Correlación temporal 4D en CUDA

https://gemini.google.com/app/3d9de90fa060b048

User prompt: Dado dos arreglos de 4 dimensiones posición y tiempo, hacer un programa c++ para hacer correlaciones con respecto a tiempo y se ejecute en cuda envidia

Response: Para calcular la correlación cruzada respecto al tiempo entre dos arreglos de 4 dimensiones (N_x, N_y, N_z, T) en **CUDA C++**, lo más aconsejable es mapear la cuadrícula espacial (x, y, z) a la estructura de bloques/hilos de la GPU y realizar el producto escalar en la dimensión temporal (t).

Concepto de Implementación

Considerando dos tensores $A(x, y, z, t)$ y $B(x, y, z, t)$, la correlación temporal (o covarianza cruzada en el origen) para cada posición espacial (x, y, z) se define como:

$$\text{Corr}(x, y, z) = \sum_{t=0}^{T-1} A(x, y, z, t) \cdot B(x, y, z, t)$$

(Nota: Si requieres calcular el coeficiente de Pearson, simplemente normalizas dividiendo entre las desviaciones estándar temporales de A y B).

Código en C++ / CUDA

El siguiente programa asigna memoria en GPU, ejecuta un kernel optimizado donde cada hilo maneja una coordenada espacial (x, y, z) y suma a lo largo del tiempo T.

```
#include <iostream>
#include <vector>
#include <cuda_runtime.h>

// Macros para manejo de errores CUDA
#define CHECK_CUDA(call) \
    do { \
        cudaError_t err = call; \
        if (err != cudaSuccess) { \
            std::cerr << "CUDA Error: " << cudaGetErrorString(err) \
                      << " en la linea " << __LINE__ << std::endl; \
            exit(EXIT_FAILURE); \
        } \
    } while (0)

// Helper para convertir índices 4D (x, y, z, t) a índice 1D plano
__host__ __device__ inline int get_index_4d(int x, int y, int z, int t,
                                            int Ny, int Nz, int T) {
    return ((x * Ny + y) * Nz + z) * T + t;
}

// Helper para convertir índices 3D (x, y, z) a índice 1D plano
__host__ __device__ inline int get_index_3d(int x, int y, int z,
                                            int Ny, int Nz) {
    return (x * Ny + y) * Nz + z;
}

// Kernel de CUDA: Cada hilo procesa un punto espacial (x, y, z) e integra sobre tiempo (T)
__global__ void temporal_correlation_kernel(const float* __restrict__ A,
                                            const float* __restrict__ B,
                                            float* __restrict__ output,
                                            int Nx, int Ny, int Nz, int T) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    int z = blockIdx.z * blockDim.z + threadIdx.z;

    // Verificar limites espacio-dimensionales
    if (x < Nx && y < Ny && z < Nz) {
```

```
            float sum = 0.0f;

            // Punteros base para evitar recalculación en el bucle
            int base_idx = get_index_4d(x, y, z, 0, Ny, Nz, T);

            // Acumulación en la dimensión temporal (T)
            for (int t = 0; t < T; ++t) {
                sum += A[base_idx + t] * B[base_idx + t];
            }

            // Guardar resultado en la matriz de salida 3D
            int out_idx = get_index_3d(x, y, z, Ny, Nz);
            output[out_idx] = sum;
        }
    }

    int main() {
        // Definición de las dimensiones
        const int Nx = 32;
        const int Ny = 32;
        const int Nz = 32;
        const int T  = 256; // Dimensión temporal

        const size_t size_4d = (size_t)Nx * Ny * Nz * T * sizeof(float);
        const size_t size_3d = (size_t)Nx * Ny * Nz * sizeof(float);

        // 1. Reservar y rellenar datos de prueba en Host (CPU)
        std::vector<float> h_A(Nx * Ny * Nz * T, 1.0f);
        std::vector<float> h_B(Nx * Ny * Nz * T, 2.0f);
        std::vector<float> h_out(Nx * Ny * Nz, 0.0f);

        // 2. Reservar memoria en Device (GPU)
        float *d_A = nullptr, *d_B = nullptr, *d_out = nullptr;
        CHECK_CUDA(cudaMalloc(&d_A, size_4d));
        CHECK_CUDA(cudaMalloc(&d_B, size_4d));
        CHECK_CUDA(cudaMalloc(&d_out, size_3d));

        // 3. Copiar datos de CPU a GPU
        CHECK_CUDA(cudaMemcpy(d_A, h_A.data(), size_4d, cudaMemcpyHostToDevice));
        CHECK_CUDA(cudaMemcpy(d_B, h_B.data(), size_4d, cudaMemcpyHostToDevice));

        // 4. Configurar la geometría de bloques e hilos CUDA
        dim3 threadsPerBlock(8, 8, 8); // 512 hilos por bloque
        dim3 blocksPerGrid((Nx + threadsPerBlock.x - 1) / threadsPerBlock.x,
                           (Ny + threadsPerBlock.y - 1) / threadsPerBlock.y,
                           (Nz + threadsPerBlock.z - 1) / threadsPerBlock.z);

        // 5. Lanzar el Kernel
        std::cout << "Ejecutando Kernel en GPU..." << std::endl;
        temporal_correlation_kernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_out, Nx, Ny, Nz, T);

        // Comprobar si hubo errores al lanzar el kernel
        CHECK_CUDA(cudaGetLastError());
        CHECK_CUDA(cudaDeviceSynchronize());

        // 6. Copiar resultados de GPU a CPU
        CHECK_CUDA(cudaMemcpy(h_out.data(), d_out, size_3d, cudaMemcpyDeviceToHost));

        // 7. Validación rápida en CPU
        std::cout << "Resultado esperado por celda espacial: " << 1.0f * 2.0f * T << std::endl;
        std::cout << "Resultado en GPU [0,0,0]: " << h_out[0] << std::endl;

        // 8. Liberar memoria GPU
        CHECK_CUDA(cudaFree(d_A));
        CHECK_CUDA(cudaFree(d_B));
        CHECK_CUDA(cudaFree(d_out));
```

```
    return 0;
}
```

Compilación y Ejecución

Para compilar el programa utilizando el compilador de NVIDIA (`nvcc`), ejecuta en la terminal:

```
nvcc -O3 -o temporal_corr temporal_corr.cu
./temporal_corr
```

Puntos clave de rendimiento:

1. **Acceso a memoria coalescente:** El arreglo está organizado de forma (x, y, z, t) haciendo que la dimensión temporal t sea adyacente en memoria. Cuando cada hilo itera secuencialmente sobre t, aprovecha al máximo la caché L1/L2 de la GPU.
2. **Dimensiones de hilos:** Usar `dim3(8, 8, 8)` asegura un tamaño de bloque de 512 hilos, aprovechando adecuadamente los *warps* (bloques de 32 hilos de NVIDIA).

User prompt: Modifica el programa CUDA para calcular la correlación de Pearson completa respecto al tiempo, incluyendo el cálculo de medias y desviaciones estándar.

Response: Para calcular el **coeficiente de correlación de Pearson** a lo largo del tiempo para cada posición espacial (x, y, z), la fórmula es:

$$r_{A,B}(x,y,z) = \frac{\sum_{t=0}^{T-1}(A_t - \mu_A)(B_t - \mu_B)}{\sum_{t=0}^{T-1}(A_t - \mu_A)^2 \quad \sum_{t=0}^{T-1}(B_t - \mu_B)^2}$$

Donde μ_A y μ_B son las medias temporales de A y B en la posición (x, y, z).

Estrategia de Implementación

Para no realizar múltiples pasadas por la memoria global (que limitaría el rendimiento en GPU), utilizamos **dos pasadas en el tiempo** dentro del mismo kernel:

1. **Primer paso ($t = 0 \dots T-1$):** Sumar los elementos de A y B para obtener las medias μ_A y μ_B.
2. **Segundo paso ($t = 0 \dots T-1$):** Calcular los desvíos $(A_t - \mu_A)$ y $(B_t - \mu_B)$, acumulando la covarianza y las varianzas en el denominador.

Código en C++ / CUDA

```
#include <iostream>
#include <vector>
#include <cmath>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) \
    do { \
        cudaError_t err = call; \
        if (err != cudaSuccess) { \
            std::cerr << "CUDA Error: " << cudaGetErrorString(err) \
                      << " en la linea " << __LINE__ << std::endl; \
            exit(EXIT_FAILURE); \
        } \
    } while (0)

// Helper para convertir índices 4D (x, y, z, t) a índice 1D plano
__host__ __device__ inline int get_index_4d(int x, int y, int z, int t,
                                            int Ny, int Nz, int T) {
    return ((x * Ny + y) * Nz + z) * T + t;
}

// Helper para convertir índices 3D (x, y, z) a índice 1D plano
__host__ __device__ inline int get_index_3d(int x, int y, int z,
```

```
                                            int Ny, int Nz) {
    return (x * Ny + y) * Nz + z;
}

// Kernel CUDA: Calcula el Coeficiente de Correlación de Pearson en T
__global__ void pearson_correlation_kernel(const float* __restrict__ A,
                                           const float* __restrict__ B,
                                           float* __restrict__ output,
                                           int Nx, int Ny, int Nz, int T) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    int z = blockIdx.z * blockDim.z + threadIdx.z;

    if (x < Nx && y < Ny && z < Nz) {
        int base_idx = get_index_4d(x, y, z, 0, Ny, Nz, T);

        // ------------------------------------------------------
        // Pasada 1: Cálculo de las Medias (\mu_A y \mu_B)
        // ------------------------------------------------------
        float sum_A = 0.0f;
        float sum_B = 0.0f;

        for (int t = 0; t < T; ++t) {
            sum_A += A[base_idx + t];
            sum_B += B[base_idx + t];
        }

        float mean_A = sum_A / T;
        float mean_B = sum_B / T;

        // ------------------------------------------------------
        // Pasada 2: Covarianza y Varianzas
        // ------------------------------------------------------
        float covariance = 0.0f;
        float var_A = 0.0f;
        float var_B = 0.0f;

        for (int t = 0; t < T; ++t) {
            float diff_A = A[base_idx + t] - mean_A;
            float diff_B = B[base_idx + t] - mean_B;

            covariance += diff_A * diff_B;
            var_A += diff_A * diff_A;
            var_B += diff_B * diff_B;
        }

        // ------------------------------------------------------
        // Cálculo final de Pearson con protección división por cero
        // ------------------------------------------------------
        float denom = sqrtf(var_A * var_B);
        float pearson_r = (denom > 1e-8f) ? (covariance / denom) : 0.0f;

        int out_idx = get_index_3d(x, y, z, Ny, Nz);
        output[out_idx] = pearson_r;
    }
}

int main() {
    const int Nx = 32;
    const int Ny = 32;
    const int Nz = 32;
    const int T  = 256;

    const size_t size_4d = (size_t)Nx * Ny * Nz * T * sizeof(float);
    const size_t size_3d = (size_t)Nx * Ny * Nz * sizeof(float);

    // 1. Inicializar datos de prueba en CPU
    std::vector<float> h_A(Nx * Ny * Nz * T);
```

```
    std::vector<float> h_B(Nx * Ny * Nz * T);
    std::vector<float> h_out(Nx * Ny * Nz, 0.0f);

    // Llenar con datos sintéticos donde B = 2*A + ruido (debería dar correlación cercana a 1.0)
    for (int i = 0; i < Nx * Ny * Nz; ++i) {
        for (int t = 0; t < T; ++t) {
            int idx = i * T + t;
            float val_a = static_cast<float>(t); // Función lineal en el tiempo
            float val_b = 2.0f * val_a + 5.0f;   // Relación perfecta + desplazamiento

            h_A[idx] = val_a;
            h_B[idx] = val_b;
        }
    }

    // 2. Reservar memoria en GPU
    float *d_A = nullptr, *d_B = nullptr, *d_out = nullptr;
    CHECK_CUDA(cudaMalloc(&d_A, size_4d));
    CHECK_CUDA(cudaMalloc(&d_B, size_4d));
    CHECK_CUDA(cudaMalloc(&d_out, size_3d));

    // 3. Copiar CPU -> GPU
    CHECK_CUDA(cudaMemcpy(d_A, h_A.data(), size_4d, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_B, h_B.data(), size_4d, cudaMemcpyHostToDevice));

    // 4. Configurar la grilla CUDA (bloques de 8x8x8 = 512 hilos)
    dim3 threadsPerBlock(8, 8, 8);
    dim3 blocksPerGrid((Nx + threadsPerBlock.x - 1) / threadsPerBlock.x,
                       (Ny + threadsPerBlock.y - 1) / threadsPerBlock.y,
                       (Nz + threadsPerBlock.z - 1) / threadsPerBlock.z);

    // 5. Ejecutar Kernel
    std::cout << "Calculando Correlación de Pearson en GPU..." << std::endl;
    pearson_correlation_kernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_out, Nx, Ny, Nz, T);

    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    // 6. Copiar resultado GPU -> CPU
    CHECK_CUDA(cudaMemcpy(h_out.data(), d_out, size_3d, cudaMemcpyDeviceToHost));

    // 7. Verificación del resultado
    std::cout << "Resultado de Pearson [0,0,0]: " << h_out[0] << " (Esperado ~1.0)" << std::endl;

    // 8. Limpieza
    CHECK_CUDA(cudaFree(d_A));
    CHECK_CUDA(cudaFree(d_B));
    CHECK_CUDA(cudaFree(d_out));

    return 0;
}
```

Optimizaciones incluidas:

1. **`sqrtf` en precisión simple:** Uso de las intrínsecas de punto flotante de 32 bits de CUDA para evitar conversiones innecesarias a `double`.
2. **Protección contra división por cero:** La evaluación `denom > 1e-8f` evita valores NaN si las señales en alguna coordenada espacial son constantes en el tiempo (varianza 0).
3. **Localidad temporal en Registros:** `mean_A` y `mean_B` se almacenan en registros rápidos de cada hilo GPU para el bucle acumulador secundario.
